

# TREES

The Complete Data Structures & Algorithms Guide  
Zero to Big Tech Interview Level

---

Concepts • Code (Python) • Complexity Analysis • Patterns  
Company-Tagged Practice Problems • Interview Playbook

Prepared for: DSA Coach (Tree) AI Agent Project

# Table of Contents

---

1. Why Trees? Real-World Motivation
2. Core Terminology
3. Types of Trees — The Big Picture
4. Binary Trees — Representation & Classification
5. Tree Traversals (Recursive, Iterative, Morris)
6. Binary Search Trees (BST)
7. Self-Balancing Trees — AVL & Red-Black
8. Heaps (as Trees) & Priority Queues
9. Tries (Prefix Trees)
10. Segment Trees
11. Fenwick Tree (Binary Indexed Tree)
12. B-Trees & B+ Trees (Database Indexing)
13. Core Interview Algorithms (LCA, Diameter, Serialize, etc.)
14. Recursion Template & DFS/BFS Decision Framework
15. Complexity Cheat Sheet
16. Company-Wise Curated Problem List
17. Real-World Applications of Trees
18. 4-Week Study Roadmap

# 1. Why Trees? Real-World Motivation

---

A tree is a **hierarchical, non-linear data structure** made of nodes connected by edges, with no cycles. Unlike arrays or linked lists (linear structures), trees model relationships where one thing has multiple 'children' — which is exactly how most real information is organized.

## Where trees run the world, quietly

- **File systems** — folders and files (Windows Explorer, Linux ext4, macOS APFS) are literally trees.
- **The DOM** — every webpage you render is a tree of HTML elements; browsers re-traverse it constantly.
- **Databases** — B+ Trees power indexes in MySQL (InnoDB), PostgreSQL, Oracle for  $O(\log n)$  lookups on billions of rows.
- **Compilers** — Abstract Syntax Trees (AST) represent your code's structure before it's compiled/executed.
- **Networking** — IP routing tables use Tries (specifically Patricia tries) for longest-prefix matching.
- **Autocomplete / spellcheck** — Google Search, IDEs use Tries for prefix lookups in  $O(\text{word length})$ .
- **Version control** — Git's object model (commits, trees, blobs) is a Merkle Tree.
- **Decision-making** — Decision Trees / Random Forests in ML; game AI (chess engines use Minimax trees).
- **Priority scheduling** — OS task schedulers and Dijkstra's algorithm use Heaps (a tree variant).
- **RAG systems** (relevant to your project!) — document chunking hierarchies, vector index structures like HNSW graphs, and hierarchical summarization ('RAPTOR') all borrow tree concepts.

■ For your DSA Coach agent project: framing tree problems with these real analogies (e.g. 'this is literally how your file explorer works') makes explanations far more memorable for the student the agent is coaching — consider baking that into your prompt design.

## 2. Core Terminology

| Term                  | Meaning                                                      |
|-----------------------|--------------------------------------------------------------|
| Node                  | A single element containing data + references to children    |
| Root                  | The topmost node; has no parent                              |
| Edge                  | The link connecting a parent to a child                      |
| Leaf                  | A node with no children                                      |
| Parent / Child        | Direct relationship one level apart                          |
| Siblings              | Nodes sharing the same parent                                |
| Ancestor / Descendant | Any node on the path up to root / down to leaves             |
| Depth of a node       | Number of edges from root to that node                       |
| Height of a node      | Number of edges on the longest path from that node to a leaf |
| Height of tree        | Height of the root node                                      |
| Degree of a node      | Number of children it has                                    |
| Subtree               | A node plus all its descendants, treated as its own tree     |
| Level                 | All nodes at the same depth                                  |

## 3. Types of Trees — The Big Picture

Before diving deep, know the map. Every specialized tree exists to optimize ONE operation at the cost of another.

| Tree Type            | Optimized For                               | Used In                                               |
|----------------------|---------------------------------------------|-------------------------------------------------------|
| Binary Tree          | General hierarchical data                   | Expression trees, general modeling                    |
| Binary Search Tree   | Ordered search $O(\log n)$ avg              | Symbol tables, ordered maps                           |
| AVL / Red-Black Tree | Guaranteed $O(\log n)$ even worst-case      | C++ <code>std::map</code> , Java <code>TreeMap</code> |
| Heap                 | Fast min/max retrieval                      | Priority queues, heapsort, Dijkstra                   |
| Trie                 | Prefix search $O(\text{word length})$       | Autocomplete, spellcheck, IP routing                  |
| Segment Tree         | Range queries + updates in $O(\log n)$      | Competitive programming, range sums                   |
| Fenwick Tree (BIT)   | Prefix sums with $O(\log n)$ update         | Frequency tables, inversions counting                 |
| B-Tree / B+ Tree     | Disk-friendly $O(\log n)$ with huge fan-out | Databases, filesystems                                |



## 4. Binary Trees — Representation & Classification

### 4.1 Node representation (Python)

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# Building a small tree manually:
#       1
#      / \
#     2   3
#    /
#   4
root = TreeNode(1)
root.left = TreeNode(2, TreeNode(4))
root.right = TreeNode(3)
```

### 4.2 Array representation (used for Heaps)

For a node at index  $i$  (0-indexed array): left child =  $2i+1$ , right child =  $2i+2$ , parent =  $(i-1)//2$ . This avoids pointer overhead entirely — critical for Heaps.

### 4.3 Classification you MUST know cold for interviews

| Type                      | Definition                                                              |
|---------------------------|-------------------------------------------------------------------------|
| Full Binary Tree          | Every node has 0 or 2 children (never exactly 1)                        |
| Complete Binary Tree      | All levels filled except possibly the last, which fills left-to-right   |
| Perfect Binary Tree       | All internal nodes have 2 children AND all leaves are at the same level |
| Balanced Binary Tree      | Height difference between left & right subtree of every node $\leq 1$   |
| Degenerate (Pathological) | Each parent has only one child — behaves like a linked list, $O(n)$ ops |

■ Interview trap: 'balanced' does NOT mean equal number of nodes on each side — it means height difference bounded by a small constant (usually 1). Height-balance, not node-count balance.

## 5. Tree Traversals

---

This is the single most important skill in the entire tree topic — nearly every tree problem is 'traversal + something extra.' There are two families: **Depth-First (DFS)** and **Breadth-First (BFS)**.

### 5.1 DFS — Preorder (Root → Left → Right)

Use when you need to process/copy a node *before* its children — e.g. serializing a tree, or building a prefix expression.

```
def preorder(root):
    if not root:
        return []
    return [root.val] + preorder(root.left) + preorder(root.right)

# Iterative with explicit stack
def preorder_iter(root):
    if not root: return []
    stack, out = [root], []
    while stack:
        node = stack.pop()
        out.append(node.val)
        if node.right: stack.append(node.right)    # push right first
        if node.left:  stack.append(node.left)     # so left is popped first
    return out
```

### 5.2 DFS — Inorder (Left → Root → Right)

For a **Binary Search Tree**, inorder traversal visits nodes in **sorted ascending order**. This single fact powers dozens of BST problems (validate BST, kth smallest, etc.)

```
def inorder(root):
    if not root:
        return []
    return inorder(root.left) + [root.val] + inorder(root.right)

def inorder_iter(root):
    stack, out, curr = [], [], root
    while stack or curr:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        out.append(curr.val)
        curr = curr.right
    return out
```

### 5.3 DFS — Postorder (Left → Right → Root)

Use when children must be fully processed before the parent — e.g. computing subtree heights, deleting a tree bottom-up, evaluating an expression tree.

```
def postorder(root):
    if not root:
        return []
```

```
return postorder(root.left) + postorder(root.right) + [root.val]
```

## 5.4 BFS — Level Order Traversal

Visits nodes level by level, using a Queue. This is how you solve almost every 'print level by level' / 'zigzag' / 'right side view' problem.

```
from collections import deque

def level_order(root):
    if not root:
        return []
    result, queue = [], deque([root])
    while queue:
        level_size = len(queue)
        level = []
        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result
```

## 5.5 Morris Traversal — O(1) space inorder (advanced)

Uses temporary 'threads' (pointers to the inorder successor) instead of a stack or recursion, achieving O(1) auxiliary space. A favorite curveball at Google/Amazon onsite interviews.

```
def morris_inorder(root):
    out, curr = [], root
    while curr:
        if not curr.left:
            out.append(curr.val)
            curr = curr.right
        else:
            pred = curr.left
            while pred.right and pred.right != curr:
                pred = pred.right
            if not pred.right:
                pred.right = curr      # create thread
                curr = curr.left
            else:
                pred.right = None      # remove thread
                out.append(curr.val)
                curr = curr.right
    return out
```

| Traversal                      | Time | Space (recursive)      | Space (Morris) |
|--------------------------------|------|------------------------|----------------|
| Preorder / Inorder / Postorder | O(n) | O(h) — recursion stack | O(1)           |
| Level Order (BFS)              | O(n) | O(w) — widest level    | N/A            |

## 6. Binary Search Trees (BST)

---

A BST maintains the invariant: for every node, all values in the **left** subtree are smaller, and all values in the **right** subtree are larger (assuming no duplicates).

### 6.1 Search — $O(h)$

```
def search(root, target):
    if not root or root.val == target:
        return root
    if target < root.val:
        return search(root.left, target)
    return search(root.right, target)
```

### 6.2 Insert — $O(h)$

```
def insert(root, val):
    if not root:
        return TreeNode(val)
    if val < root.val:
        root.left = insert(root.left, val)
    else:
        root.right = insert(root.right, val)
    return root
```

### 6.3 Delete — $O(h)$ — the classic 3-case interview question

Deletion has 3 cases: (1) node is a leaf → just remove it. (2) node has one child → replace node with that child. (3) node has two children → replace its value with the **inorder successor** (smallest value in right subtree), then delete that successor.

```
def delete(root, key):
    if not root:
        return None
    if key < root.val:
        root.left = delete(root.left, key)
    elif key > root.val:
        root.right = delete(root.right, key)
    else:
        if not root.left: return root.right
        if not root.right: return root.left
        # two children: find inorder successor (min of right subtree)
        succ = root.right
        while succ.left:
            succ = succ.left
        root.val = succ.val
        root.right = delete(root.right, succ.val)
    return root
```

| Operation | Average (balanced) | Worst case (skewed) |
|-----------|--------------------|---------------------|
| Search    | $O(\log n)$        | $O(n)$              |
| Insert    | $O(\log n)$        | $O(n)$              |

|        |             |        |
|--------|-------------|--------|
| Delete | $O(\log n)$ | $O(n)$ |
|--------|-------------|--------|

■ Worst case happens when you insert sorted data into a plain BST — it degenerates into a linked list. This is exactly WHY self-balancing trees (AVL, Red-Black) exist.

## 7. Self-Balancing Trees — AVL & Red-Black

---

### 7.1 AVL Tree

An AVL tree is a BST where, for every node, the **balance factor** ( $\text{height}(\text{left}) - \text{height}(\text{right})$ ) is always -1, 0, or +1. After every insert/delete, rotations restore this invariant.

#### The 4 rotation cases

- **Left-Left (LL)**: imbalance from left child's left subtree → single **right** rotation
- **Right-Right (RR)**: imbalance from right child's right subtree → single **left** rotation
- **Left-Right (LR)**: left child's right subtree is heavy → left rotate the child, then right rotate root
- **Right-Left (RL)**: right child's left subtree is heavy → right rotate the child, then left rotate root

```
def right_rotate(y):
    x = y.left
    y.left = x.right
    x.right = y
    return x # new subtree root

def left_rotate(x):
    y = x.right
    x.right = y.left
    y.left = x
    return y # new subtree root

def get_height(n):
    return n.height if n else 0

def get_balance(n):
    return get_height(n.left) - get_height(n.right) if n else 0

def avl_insert(node, val):
    if not node:
        return AVLNode(val)
    if val < node.val:
        node.left = avl_insert(node.left, val)
    else:
        node.right = avl_insert(node.right, val)

    node.height = 1 + max(get_height(node.left), get_height(node.right))
    balance = get_balance(node)

    if balance > 1 and val < node.left.val:           # LL
        return right_rotate(node)
    if balance < -1 and val > node.right.val:         # RR
        return left_rotate(node)
    if balance > 1 and val > node.left.val:           # LR
        node.left = left_rotate(node.left)
        return right_rotate(node)
    if balance < -1 and val < node.right.val:         # RL
        node.right = right_rotate(node.right)
        return left_rotate(node)

    return node
```

## 7.2 Red-Black Tree (concept-level — rarely hand-coded in interviews)

A Red-Black tree colors every node red or black and enforces 5 rules (root is black, no two red nodes in a row, every path from a node to its descendant leaves has the same number of black nodes, etc.) to guarantee  $\text{height} \leq 2 \cdot \log(n+1)$ . It allows more rotations to be avoided than AVL, trading slightly worse search speed for faster inserts/deletes — which is why Java's `TreeMap/TreeSet` and C++'s `std::map` use Red-Black trees internally rather than AVL.

|                     | AVL Tree                                 | Red-Black Tree                                   |
|---------------------|------------------------------------------|--------------------------------------------------|
| Balance strictness  | Stricter ( $\text{height diff} \leq 1$ ) | Looser ( $\text{height} \leq 2 \times \log n$ )  |
| Search speed        | Faster (more balanced)                   | Slightly slower                                  |
| Insert/Delete speed | Slower (more rotations)                  | Faster (fewer rotations)                         |
| Real-world use      | Databases needing fast reads             | Language libraries (Java/C++ maps), Linux kernel |

## 8. Heaps (as Trees) & Priority Queues

A Binary Heap is a **complete binary tree** stored in an array, satisfying the heap property: in a Max-Heap, every parent  $\geq$  its children (root = maximum); in a Min-Heap, every parent  $\leq$  its children (root = minimum).

```
import heapq

min_heap = []
heapq.heappush(min_heap, 5)
heapq.heappush(min_heap, 1)
heapq.heappush(min_heap, 3)
print(heapq.heappop(min_heap))  # 1 (smallest)

# Python only has min-heap; simulate max-heap by negating values
max_heap = []
for v in [5, 1, 3]:
    heapq.heappush(max_heap, -v)
print(-heapq.heappop(max_heap))  # 5 (largest)
```

### Manual heapify (how heapq works under the hood)

```
def sift_down(arr, i, n):
    smallest = i
    left, right = 2*i + 1, 2*i + 2
    if left < n and arr[left] < arr[smallest]:
        smallest = left
    if right < n and arr[right] < arr[smallest]:
        smallest = right
    if smallest != i:
        arr[i], arr[smallest] = arr[smallest], arr[i]
        sift_down(arr, smallest, n)

def build_min_heap(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        sift_down(arr, i, n)
```

| Operation                         | Time Complexity                                 |
|-----------------------------------|-------------------------------------------------|
| Build heap from array             | $O(n)$ (not $O(n \log n)$ — amortized analysis) |
| Push / Pop (insert / extract-min) | $O(\log n)$                                     |
| Peek min/max                      | $O(1)$                                          |
| Heapsort (full sort using a heap) | $O(n \log n)$                                   |

■ Classic heap interview problems: 'Kth Largest Element in an Array', 'Merge K Sorted Lists', 'Top K Frequent Elements', 'Find Median from Data Stream' (uses TWO heaps).

# 9. Tries (Prefix Trees)

A Trie stores strings character-by-character along paths from the root, so all words sharing a prefix share the same path. This gives  $O(\text{word length})$  lookup regardless of how many words are stored — dramatically faster than scanning a word list.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end = True

    def search(self, word):
        node = self._find(word)
        return node is not None and node.is_end

    def starts_with(self, prefix):
        return self._find(prefix) is not None

    def _find(self, s):
        node = self.root
        for ch in s:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node
```

| Operation               | Time   | Space                        |
|-------------------------|--------|------------------------------|
| Insert word of length L | $O(L)$ | $O(L)$ worst case (new path) |
| Search / Prefix search  | $O(L)$ | $O(1)$ extra                 |

■ Real-world: Google Search autocomplete, IDE autocomplete, T9 predictive texting, IP routing tables (as a binary trie on bits), and spell-checkers all run on tries.

## 10. Segment Trees

A Segment Tree answers **range queries** (sum, min, max, GCD over a range) AND supports **point updates**, both in  $O(\log n)$  — something a plain array or prefix-sum array cannot do efficiently together (prefix sums are  $O(1)$  query but  $O(n)$  update).

```
class SegmentTree:
    def __init__(self, arr):
        self.n = len(arr)
        self.tree = [0] * (4 * self.n)
        self._build(arr, 0, 0, self.n - 1)

    def _build(self, arr, node, start, end):
        if start == end:
            self.tree[node] = arr[start]
            return
        mid = (start + end) // 2
        self._build(arr, 2*node+1, start, mid)
        self._build(arr, 2*node+2, mid+1, end)
        self.tree[node] = self.tree[2*node+1] + self.tree[2*node+2]

    def query(self, l, r):
        # sum on [l, r]
        return self._query(0, 0, self.n - 1, l, r)

    def _query(self, node, start, end, l, r):
        if r < start or end < l:
            return 0
        if l <= start and end <= r:
            return self.tree[node]
        mid = (start + end) // 2
        return (self._query(2*node+1, start, mid, l, r) +
                self._query(2*node+2, mid+1, end, l, r))

    def update(self, idx, val):
        self._update(0, 0, self.n - 1, idx, val)

    def _update(self, node, start, end, idx, val):
        if start == end:
            self.tree[node] = val
            return
        mid = (start + end) // 2
        if idx <= mid:
            self._update(2*node+1, start, mid, idx, val)
        else:
            self._update(2*node+2, mid+1, end, idx, val)
        self.tree[node] = self.tree[2*node+1] + self.tree[2*node+2]
```

| Operation    | Time        | Space                |
|--------------|-------------|----------------------|
| Build        | $O(n)$      | $O(4n) \approx O(n)$ |
| Range Query  | $O(\log n)$ | —                    |
| Point Update | $O(\log n)$ | —                    |

## 11. Fenwick Tree (Binary Indexed Tree)

---

A lighter-weight alternative to Segment Trees, specifically for **prefix sums with updates**. Uses bit manipulation on the index to move between related array slots — much smaller code, smaller constant factor, but less flexible (mainly sums, not arbitrary range min/max).

```
class FenwickTree:
    def __init__(self, n):
        self.n = n
        self.tree = [0] * (n + 1)

    def update(self, i, delta):          # add `delta` at index i (1-indexed)
        while i <= self.n:
            self.tree[i] += delta
            i += i & (-i)                # move to next responsible index

    def prefix_sum(self, i):             # sum of [1, i]
        s = 0
        while i > 0:
            s += self.tree[i]
            i -= i & (-i)                # move to parent
        return s

    def range_sum(self, l, r):
        return self.prefix_sum(r) - self.prefix_sum(l - 1)
```

■ The line `i & (-i)` isolates the lowest set bit — this single bit trick is the entire magic behind BIT/Fenwick trees. Classic use: counting inversions in an array in  $O(n \log n)$ .

## 12. B-Trees & B+ Trees (Database Indexing)

Unlike binary trees (max 2 children), a B-Tree of order  $m$  allows each node up to  $m$  children and multiple keys per node. This wide 'fan-out' means the tree stays very **shallow** even with millions of entries — critical because each node maps to a disk page, and disk reads are ~100,000x slower than RAM reads. Fewer levels = fewer disk seeks.

- **B-Tree**: keys and data can live in internal nodes AND leaves.
- **B+ Tree**: ALL actual data lives in leaf nodes only; internal nodes are pure routing/index keys, and leaves are linked in a list — enabling fast range scans. This is what MySQL InnoDB, PostgreSQL, and most filesystems (NTFS, ext4, APFS) actually use.

| Structure                                   | Height for 1 million keys (approx) |
|---------------------------------------------|------------------------------------|
| Plain BST (unbalanced, worst case)          | up to 1,000,000                    |
| AVL / Red-Black Tree                        | ~20                                |
| B-Tree / B+ Tree (order ~500, DB page size) | ~3                                 |

■ *This is WHY databases don't just use a Red-Black tree: a 500-way B+ Tree turns a query that would need 20 disk seeks into one needing only 3.*

## 13. Core Interview Algorithms

---

### 13.1 Maximum Depth / Height

```
def max_depth(root):
    if not root:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

### 13.2 Check if Balanced (bottom-up, $O(n)$ — not $O(n^2)$ !)

```
def is_balanced(root):
    def height(node):
        if not node:
            return 0
        lh = height(node.left)
        if lh == -1: return -1
        rh = height(node.right)
        if rh == -1: return -1
        if abs(lh - rh) > 1: return -1
        return 1 + max(lh, rh)
    return height(root) != -1
```

■ Naive approach recomputes height() repeatedly at every node =  $O(n^2)$ . The trick above returns -1 as a sentinel for 'already unbalanced' to short-circuit in a single  $O(n)$  pass. Interviewers specifically probe for this optimization.

### 13.3 Diameter of a Binary Tree

The diameter is the longest path between any two nodes (may or may not pass through the root). Compute it while computing height, in one pass.

```
def diameter(root):
    best = [0]
    def height(node):
        if not node:
            return 0
        lh = height(node.left)
        rh = height(node.right)
        best[0] = max(best[0], lh + rh)    # path through this node
        return 1 + max(lh, rh)
    height(root)
    return best[0]
```

### 13.4 Lowest Common Ancestor (LCA)

```
def lca(root, p, q):
    if not root or root == p or root == q:
        return root
    left = lca(root.left, p, q)
    right = lca(root.right, p, q)
    if left and right:
        return root    # p and q found in different subtrees -> this is LCA
    return left or right
```

## 13.5 Validate a Binary Search Tree

Common mistake: only comparing a node to its immediate children. Must pass down a valid (min, max) RANGE.

```
def is_valid_bst(root, low=float('-inf'), high=float('inf')):
    if not root:
        return True
    if not (low < root.val < high):
        return False
    return (is_valid_bst(root.left, low, root.val) and
            is_valid_bst(root.right, root.val, high))
```

## 13.6 Serialize & Deserialize a Binary Tree

```
def serialize(root):
    vals = []
    def dfs(node):
        if not node:
            vals.append('#')
            return
        vals.append(str(node.val))
        dfs(node.left)
        dfs(node.right)
    dfs(root)
    return ','.join(vals)

def deserialize(data):
    vals = iter(data.split(','))
    def dfs():
        val = next(vals)
        if val == '#':
            return None
        node = TreeNode(int(val))
        node.left = dfs()
        node.right = dfs()
        return node
    return dfs()
```

## 13.7 Kth Smallest Element in a BST

```
def kth_smallest(root, k):
    stack, curr = [], root
    while stack or curr:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        k -= 1
        if k == 0:
            return curr.val
        curr = curr.right
```

## 13.8 Right Side View (BFS pattern)

```
from collections import deque
def right_side_view(root):
```

```

if not root: return []
out, queue = [], deque([root])
while queue:
    size = len(queue)
    for i in range(size):
        node = queue.popleft()
        if i == size - 1:
            out.append(node.val)      # last node in level = visible from right
        if node.left: queue.append(node.left)
        if node.right: queue.append(node.right)
return out

```

## 13.9 Path Sum (root-to-leaf)

```

def has_path_sum(root, target):
    if not root:
        return False
    if not root.left and not root.right:
        return root.val == target
    remaining = target - root.val
    return (has_path_sum(root.left, remaining) or
            has_path_sum(root.right, remaining))

```

# 14. Recursion Template & DFS/BFS Decision Framework

## 14.1 The universal tree recursion template

```
def solve(node):
    # 1. Base case
    if not node:
        return BASE_VALUE

    # 2. Recurse on children first (post-order style)
    left_result = solve(node.left)
    right_result = solve(node.right)

    # 3. Combine results using node.val
    return combine(node.val, left_result, right_result)
```

90% of tree problems fit this exact skeleton. The hard part is always defining what *BASE\_VALUE* and *combine()* should be for the specific problem.

## 14.2 When to use DFS vs BFS

| Signal in the problem                                                | Use                          |
|----------------------------------------------------------------------|------------------------------|
| 'root-to-leaf path', 'ancestor', height/depth, backtracking          | DFS (recursion or stack)     |
| 'level by level', 'shortest path in unweighted tree', 'nearest node' | BFS (queue)                  |
| Need to process ALL nodes at the same depth together                 | BFS                          |
| Need to go as deep as possible before backtracking                   | DFS                          |
| Space is a concern and tree is very wide (many nodes per level)      | DFS (uses O(h) not O(width)) |
| Space is a concern and tree is very deep (risk of stack overflow)    | BFS (or iterative DFS)       |

## 15. Complexity Cheat Sheet

| Structure / Operation                   | Average                    | Worst Case                 | Space                                          |
|-----------------------------------------|----------------------------|----------------------------|------------------------------------------------|
| BST — Search/Insert/Delete              | $O(\log n)$                | $O(n)$                     | $O(n)$                                         |
| AVL / Red-Black — Search/Insert/Delete  | $O(\log n)$                | $O(\log n)$                | $O(n)$                                         |
| Heap — Push/Pop                         | $O(\log n)$                | $O(\log n)$                | $O(n)$                                         |
| Heap — Build from array                 | $O(n)$                     | $O(n)$                     | $O(n)$                                         |
| Trie — Insert/Search (len L)            | $O(L)$                     | $O(L)$                     | $O(\text{alphabet} \times \text{total chars})$ |
| Segment Tree — Build                    | $O(n)$                     | $O(n)$                     | $O(n)$                                         |
| Segment Tree — Query/Update             | $O(\log n)$                | $O(\log n)$                | $O(n)$                                         |
| Fenwick Tree — Update/Query             | $O(\log n)$                | $O(\log n)$                | $O(n)$                                         |
| B-Tree (order m) — Search/Insert/Delete | $O(\log_{\blacksquare} n)$ | $O(\log_{\blacksquare} n)$ | $O(n)$                                         |
| Any tree traversal (DFS/BFS)            | $O(n)$                     | $O(n)$                     | $O(h)$ or $O(\text{width})$                    |

## 16. Company-Wise Curated Problem List

---

These are patterns most frequently reported across interview experience aggregators for tree questions at major tech companies. Use these to structure your DSA Coach agent's practice sets.

### Google

- Serialize and Deserialize Binary Tree
- Lowest Common Ancestor of a Binary Tree
- Binary Tree Maximum Path Sum
- Vertical Order Traversal of a Binary Tree
- Construct Binary Tree from Preorder and Inorder Traversal

### Amazon

- Validate Binary Search Tree
- Kth Smallest Element in a BST
- Binary Tree Level Order Traversal / Zigzag Level Order
- Diameter of Binary Tree
- Convert Sorted Array to Balanced BST

### Meta (Facebook)

- Binary Tree Right Side View
- Path Sum II (all root-to-leaf paths)
- Populating Next Right Pointers in Each Node
- Flatten Binary Tree to Linked List
- Sum Root to Leaf Numbers

### Microsoft

- Symmetric Tree (mirror check)
- Balanced Binary Tree check
- Invert/Mirror a Binary Tree
- Trim a Binary Search Tree
- Recover Binary Search Tree (two swapped nodes)

### Apple

- Binary Tree Inorder Traversal (iterative)
- Count Complete Tree Nodes (optimized, better than  $O(n)$ )
- Merge Two Binary Trees
- Cousins in Binary Tree

## Uber / Ola (India-heavy interview stacks)

- Top View / Bottom View of a Binary Tree
- Boundary Traversal of Binary Tree
- Diagonal Traversal
- Check if two trees are identical

■ *For your AI agent: tagging each practice problem in your RAG knowledge base with {company, pattern, difficulty, prerequisite-concepts} lets the agent adaptively recommend the next question based on what the student already solved — this is a strong feature to highlight in your architecture doc.*

## 17. Real-World Applications of Trees (Deep Dive)

---

### Huffman Coding (compression)

Builds an optimal binary tree bottom-up using a min-heap: repeatedly merge the two least-frequent symbols into a new subtree until one tree remains. Used in ZIP, JPEG, MP3, and gzip compression.

### Decision Trees (Machine Learning)

Each internal node is a feature-based split (e.g. 'age > 30?'), each leaf is a prediction. Random Forests average many decision trees; Gradient Boosted Trees (XGBoost, LightGBM) power most tabular-data ML in production today.

### Game AI — Minimax / Game Trees

Chess and Go engines build a tree of possible future moves and use Minimax with Alpha-Beta pruning to decide the best move, effectively searching the tree while cutting off provably irrelevant branches.

### Merkle Trees (Git, Blockchain)

Every leaf is a hash of a data block; every parent is a hash of its children's hashes. This lets Git and blockchains verify huge datasets by comparing a single root hash instead of every byte.

### RAG / LLM Systems (relevant to your project)

Hierarchical document summarization (e.g. the RAPTOR technique) builds a tree where leaves are raw text chunks and each parent node is an LLM-generated summary of its children — letting a retrieval system query at the right level of abstraction instead of only raw chunks.

## 18. 4-Week Study Roadmap

---

### Week 1 — Foundations

- Terminology + all 3 DFS traversals (recursive & iterative) + BFS level order
- Solve: Max Depth, Invert Tree, Same Tree, Symmetric Tree, Path Sum

### Week 2 — BST Mastery

- BST insert/search/delete, Validate BST, Kth Smallest, LCA in BST
- Solve 8-10 BST-tagged problems on your practice platform of choice

### Week 3 — Balanced Trees, Heaps, Tries

- Understand AVL rotations conceptually (rarely hand-coded live, but explained verbally)
- Heap: Kth Largest, Merge K Sorted Lists, Top K Frequent
- Trie: Implement Trie, Word Search II, Design Add-and-Search Word

### Week 4 — Advanced Structures + Mock Interviews

- Segment Tree & Fenwick Tree for range-query problems (esp. if targeting competitive programming)
- Serialize/Deserialize, Diameter, Maximum Path Sum, Vertical Order Traversal
- 2-3 timed mock interviews focused purely on trees

---

*End of document — built for the DSA Coach (Tree) AI Agent project.*